



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

Week 5 : Routing & Navigation in React

Muneeb Ur Rehman

<https://github.com/955muneeb/Glaxit-internship/tree/main/week-5/week5-react-app/week5-react-app>

Laboratory Report – Week 5, Day 4

Routing & Navigation in React

Abstract

This laboratory exercise focused on implementing **routing and navigation** in a React application using the **react-router-dom** library. Until this point, the application contained only a single page. In this lab, multiple pages were created, including **Home**, **About**, and a **404 Not Found** page. We learned how to navigate between pages without refreshing the browser and how to display a custom page when a user enters an invalid URL. This exercise introduced the concept of **client-side routing**, which is widely used in modern React applications.

Objectives

The objectives of this lab were:

- To understand client-side routing in React.
 - To learn how to use the **react-router-dom** library.
 - To configure **BrowserRouter**, **Routes**, and **Route**.
 - To create navigation using **NavLink**.
 - To implement a custom **404 Not Found** page.
 - To understand the difference between traditional page navigation and client-side routing.
-

Introduction to Routing

Most modern websites contain multiple pages such as **Home**, **About**, **Contact**, and **Services**. In traditional websites, clicking a link sends a new request to the server, causing the entire webpage to reload. This process can be slower and may reset the application's current state.

React applications use **client-side routing**, where navigation happens inside the browser without reloading the entire page. Instead of requesting a new webpage from the server, React simply changes the component displayed on the screen. This makes navigation much faster and provides a smoother user experience.

To enable this feature, React uses the **react-router-dom** library.

Task Performed

The task was to create a React application with multiple pages.

The following pages were added:

- **Home Page**
- **About Page**
- **404 Not Found Page**

Navigation links were created so users could move between pages without refreshing the browser. A custom 404 page was also implemented to handle invalid URLs.

Step-by-Step Implementation

Step 1 – Setting Up BrowserRouter

The entire application was wrapped inside the **BrowserRouter** component in **main.jsx**.

BrowserRouter enables all routing features in the application and keeps track of the current URL. Without it, React Router cannot perform navigation between pages.

Step 2 – Creating Routes

Inside **App.jsx**, the **Routes** component was used to organize all application routes.

Each **Route** connected a specific URL path with a React component.

For example:

- `/` displays the **Home** page.
- `/about` displays the **About** page.

When a user visits one of these URLs, React automatically displays the corresponding component.

Step 3 – Creating the 404 Page

A separate **NotFound** component was created to handle invalid URLs.

A route with the wildcard path `*` was added at the end of the routing list.

Whenever a user enters a URL that does not match any existing route, React displays the custom **404 Not Found** page instead of showing a blank screen or an error.

This improves the user experience by informing users that the requested page does not exist.

Step 4 – Building Navigation

A navigation bar was created using **NavLink**.

Unlike a normal HTML `<a>` tag, **NavLink** allows navigation without refreshing the browser.

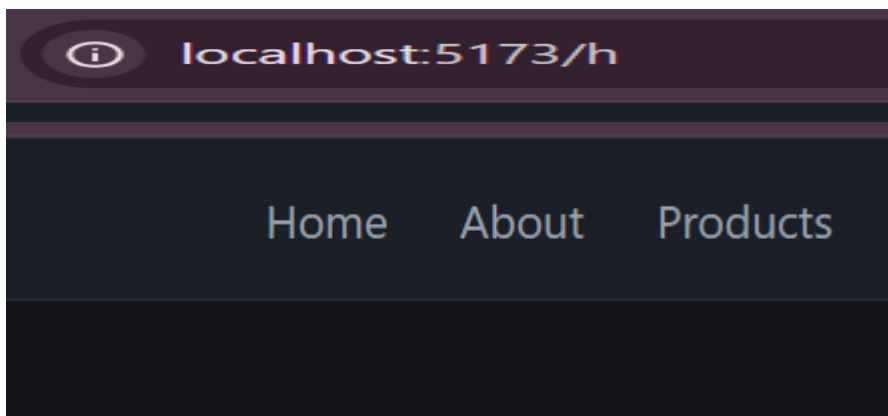
Another advantage of **NavLink** is that it automatically highlights the active page. This helps users know which page they are currently viewing.

The navigation links connected the Home and About pages, allowing smooth movement between different sections of the application.

Step 5 – Testing the Application

The application was tested in different ways to ensure everything worked correctly.

- Clicking the navigation links successfully switched between pages.
- The browser did not perform a full page reload.
- Typing an invalid URL displayed the custom **404 Not Found** page.
- The active navigation link was highlighted correctly.



404

This page doesn't exist.

[← Back home](#)

© 2026 Week5.dev. All rights reserved.